Vim: Text Editor

We must learn at least one text editor that can be used from a text-only shell prompt and be confident in editing text-based configuration files from a terminal window, through remote SSH logins, or via a web console. Learning **vim** is a great choice, as it enables you to edit files on a server without a GUI.

Vim is almost always installed in the server. If not, install using your respective package manager.

1. Update the system and install vim

```
→ sudo pacman -Suyy && sudo pacman -S vim
```

2. open a file for editing using following command:

```
→ vim filename
```

The **vim** editor works with data in a memory buffer. If you start vim without specifying a filename, or if the file doesn't exist, vim opens a new buffer area for editing. If you specify an existing file on the command line, vim reads the entire file's contents into a buffer. Any changes made in the buffer are written to the file only when you run the write command.

Operation Modes in Vim

Vim has several modes of operation including **command mode**, **insert mode**, **visual mode**, and **extended command mode**.

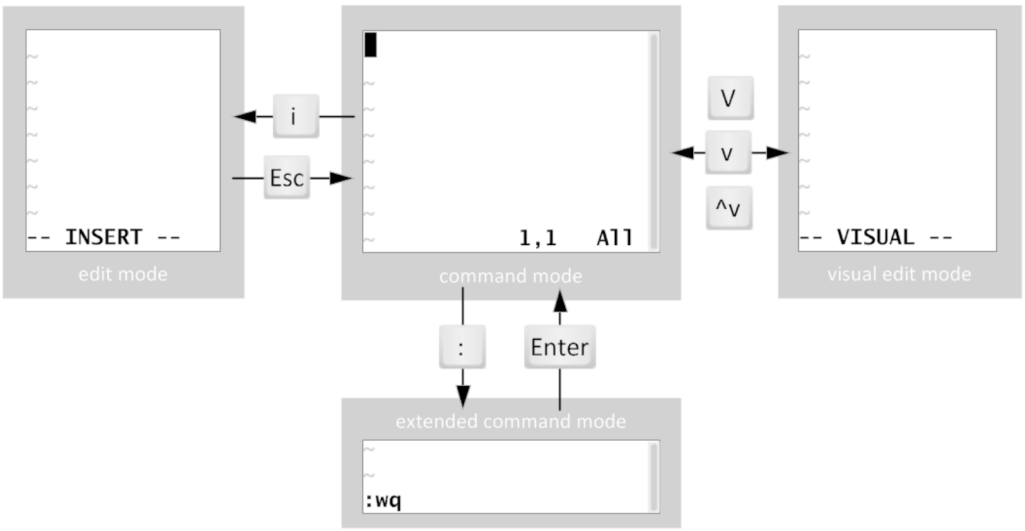


Figure 1: Operation modes in vim

When you first open Vim, it starts in c**ommand mode,** which is used for navigation and other text manipulation.

Access specific editing functionality:

1. An **i** keystroke enters **insert mode,** where all text typed becomes file content. Pressing **Esc** returns to **command mode.**
2. A **:** keystroke begins **extended command mode** for tasks such as writing the file and quitting the Vim editor.
3. A **v** keystroke enters **visual mode,** where multiple characters may be selected for text manipulation. **Shift+V** is used for multiline and **Ctrl+V** for block selection. The same keystroke used to enter visual mode (**v, Shift+V or Ctrl+V)** is used to exit.

If you get lost, just press **Esc** to get in **command mode.**

Insert Mode

Command	Description
`i`	Insert text **before** the current cursor (Enter Insert Mode)
`Esc`	Return to **Command Mode** from Insert or other modes

Command Mode

Navigation:

To move cursor you can use arro keys or following keys:

Command	Description
`h`	Move cursor left one character
`j`	Move cursor down one line
`k`	Move cursor up one line
`l`	Move cursor right one character

To move around within large text files

Command	Description
`PageDown` / `Ctrl+F`	Move forward one screen of data
`PageUp` / `Ctrl+B`	Move backward one screen of data
`G`	Move to the last line in the buffer
`numG`	Move to the line number `num` in the buffer
`gg`	Move to the first line in the buffer

Text Editing Manipulation:

Command	Description
`x`	Deletes the character at the current cursor position
`dd`	Deletes the entire line at the current cursor position
`dw`	Deletes the word at the current cursor position
`d$`	Deletes from the current position to the end of the line
`J`	Joins the current line with the next one (removes line break)
`u`	Undoes the previous edit command

Text Appending and Replacing:

Command	Description
`a`	Appends data **after** the current cursor position
`A`	Appends data **at the end of the line** at the current cursor position
`r`	Replaces the character at the cursor with a specified character
`R`	Overwrites text at the cursor position until you press `Esc`

Copying and Pasting:

Command	Description
`y`	Yank (copy) the selected text
`p`	Paste the yanked or deleted text after the cursor

Searching String

Command	Description
`/string_to_search`	Searches **forward** for the specified string in the file

---

Extended Command Mode

File Commands:

Command	Description
`:q!`	Quit without saving changes
`:w`	Write (save) the current file
`:wq`	Write and quit
`:x`	Write if changes were made, then quit
`:q`	Quit (only if no unsaved changes)
`:w <file>`	Write the current file as a different file (save as)
`:e <file>`	Edit (open) a different file
`:n`	Move to the next file in the argument list
`:prev`	Move to the previous file in the argument list

Buffer Window Management:

Command	Description
`:ls or :buffers`	List open buffers
`:b <n>`	Switch to buffer number n
`:bd`	Delete a buffer
`:sp`	Open a file in a horizontal split
`:vsp`	Open a file in a vertical split
`:tabnew`	Open a file in a new tab

Search and Replace:

Command	Description
`:s/old/new/`	Replace first occurrence on the current line
`:s/old/new/g`	Replace all occurrences on the current line
`:%s/old/new/g`	Replace all in the entire file
`:%s/old/new/gc`	Replace with confirmation

Navigation:

Command	Description
`:1`	Go to line 1
`:n`	Go to line number n
`:$`	Go to the last line
`:/pattern`	Search for pattern

Configuration & Settings:

Command	Description
`:set number`	Show line numbers
`:set nonumber`	Hide line numbers
`:set paste`	Paste mode (disables auto-indent)
`:set ignorecase`	Case-insensitive search
`:syntax on`	Enable syntax highlighting
`:colorscheme <name>`	Change color scheme

Others:

Command	Description
`:!<cmd>`	Run a shell command (e.g. :!ls)
`:source <file>`	Source a Vim script or config file
`:history`	Show command history

Command	Description
`:echo <expr>`	Evaluate and show a Vim expression

Visual Mode

Command	Description
`v`	Enter **Character-wise** Visual Mode
`V`	Enter **Line-wise** Visual Mode
`Ctrl+v`	Enter **Block-wise** Visual Mode (Visual Block)

Below are commonly used **commands in Visual mode** for each of these:

Navigation (Works in all Visual modes):

Command(s)	Description
`h`, `j`, `k`, `l`	Move cursor left, down, up, right
`w`, `e`, `b`	Move forward by word, to end of word, back by word
`0`, `^`, `$`	Start of line, first non-blank character, end of line
`gg`, `G`	Go to start (`gg`) or end (`G`) of the file

Common Commands (after selecting in any Visual mode):

Command	Description
`d`	Delete selected text
`y`	Yank (copy) selected text
`c`	Change (delete + insert)
`~`	Toggle case of selected text
`u`	Lowercase selected text
`U`	Uppercase selected text
`>`	Indent selection right
`<`	Indent selection left
`=`	Auto-indent selection
`:`	Run an Ex command on selected lines (:'<,'>s/foo/bar/g)
`gU`	Uppercase selection (alternate)
`gu`	Lowercase selection (alternate)
`J`	Join selected lines
`!`	Filter through external command (!sort to sort lines)
